Lab 07 – Lists, Tuples, Dictionaries, Sets
CLO Alignment: CLO2
Topic: Python Data Structures
Learning Objectives
After completing this lab, students can:

- ✓ Use list, tuple, dictionary, and set
- ✓ Store and process multiple values
- ✓ Search, update, delete data
- ✓ Apply suitable data structures for real problems
- ✓ Build small data management programs

Importance Notes:

Data Structure	Suitable Use
List	ordered collection, grades, product list
Tuple	fixed data, student profile, coordinates
Dictionary	key-value data, student records
Set	unique values, duplicate removal, course enrollment

Exercise 1 – Student Grade List
Problem Statement
Create a program to store student grades and calculate:

- total number of grades
- average grade
- highest grade
- lowest grade
- passed students
- failed students

Passing condition: grade >= 50

Exercise 2 – Student Information
Problem Statement
Use a tuple to store student information: Student ID, Name, Major, Year

Why Tuple?
A tuple is suitable when data should not be changed.

Exercise 3 – Student Grade Management System

Problem Statement

Create a **dictionary** to store student information.

Each student has:

- ✓ student ID
- ✓ name
- ✓ grade

- Create dictionary
- Display all students
- Search student by ID
- Update student grad

Exercise 4 – Unique Skills

Problem Statement

Given a list of student skills, remove duplicated skills.

Input: skills = ["Python", "Java", "Python", "SQL", "Java", "HTML"]

Expected output: {'Python', 'Java', 'SQL', 'HTML'}

Exercise 5 – Count Words in a Sentence

Problem Statement

Input a sentence and count how many times each word appears.

Example: Input "python is easy python is powerful"

Expected output:

- ✓ python: 2
- ✓ is: 2
- ✓ easy: 1
- ✓ powerful: 1

Exercise 6 – Product Inventory System

Problem Statement

Create a simple inventory system.

Each product has:

- ✓ product name
- ✓ quantity
- ✓ price

Required features:

- ✓ display all products

- ✓ add new product
- ✓ update quantity
- ✓ calculate total inventory value

Exercise 1 – 6: (0.5 mark)

Exercise 7 – Menu-Based Student Management System

Requirement

Build a menu system:

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add student
2. View all students
3. Search student
4. Update grade
5. Delete student
6. Show statistics
7. Exit
```

Exercise 8 – Inventory System with Low Stock Warning

Requirement

Extend the inventory system.

Add:

- ✓ product category
- ✓ low stock warning
- ✓ search by category
- ✓ calculate value by category

Low stock condition: quantity < 5

Exercise 9 – Remove Duplicate Students Using Set

Problem Statement

A class has duplicated student IDs:

student_ids = ["SE001", "SE002", "SE003", "SE001", "SE002", "SE004"]

Tasks:

- ✓ remove duplicates
- ✓ count unique students
- ✓ find duplicated IDs

Exercise 10 – Course Enrollment System

Problem Statement

Use sets to manage students enrolled in courses.
Tasks:

- ✓ students enrolled in both courses
- ✓ students only in Python course
- ✓ students only in Data course
- ✓ all unique students

Exercise 11 – Student Ranking System

Problem Statement

Example: input student data following:

students = {
"SE001": {"name": "John", "grade": 85},
"SE002": {"name": "Anna", "grade": 92},
"SE003": {"name": "Peter", "grade": 76},
"SE004": {"name": "Mary", "grade": 88}

Tasks:

- ✓ sort students by grade descending
- ✓ display ranking
- ✓ classify students

Classification:

Grade	Classification
>= 90	Excellent
>= 80	Good
>= 65	Average
else	Weak

Exercise 12 – Course Registration Conflict Checker

Problem Statement

A student wants to register for multiple courses. Each course has a schedule.
Use dictionary and set to detect schedule conflicts.

Requirements

Students must:

- ✓ input selected course codes
- ✓ check whether any selected courses have overlapping slots
- ✓ display conflicting courses and conflict slots

Example:

```
courses = {
"PDS301m": {"name": "Python for Data Science", "slots": {"Mon-1", "Wed-2"}},
"SWE201c": {"name": "Software Engineering", "slots": {"Mon-1", "Fri-3"}},
"DBI202": {"name": "Database Systems", "slots": {"Tue-2", "Thu-4"}},
"PRN212": {"name": "C# Programming", "slots": {"Wed-2", "Sat-1"}}
}
```

Expected Output:

```
Schedule conflicts found:
PDS301m conflicts with SWE201c at {'Mon-1'}
PDS301m conflicts with PRN212 at {'Wed-2'}
```

Exercise 13: Mini Recommendation System Using Dictionary and Set

Problem Statement

Build a simple course recommendation system based on students' learned skills.
Each course requires some prerequisite skills.
The student already has a set of skills.
Recommend courses where the student satisfies all prerequisites.

Requirements

You must:

- ✓ compare student skills with course requirements
- ✓ recommend suitable courses
- ✓ show missing skills for unsuitable courses

Example:

```
student_skills = {"Python", "Basic Math", "Excel"}

courses = {
"Data Analysis": {"Python", "Basic Math"},
"Machine Learning": {"Python", "Statistics", "Linear Algebra"},
"Web Scraping": {"Python", "HTML"},
"Business Intelligence": {"Excel", "SQL"},
"Data Visualization": {"Python", "Excel"}
}
```

Expected Output:

```
===== RECOMMENDED COURSES =====
- Data Analysis
- Data Visualization

===== COURSES NOT READY YET =====
Machine Learning - Missing skills: {'Statistics', 'Linear Algebra'}
```

Web Scraping - Missing skills: {'HTML'}
Business Intelligence - Missing skills: {'SQL'}